CTF Writeup

Echo Signature — CTF Writeup

Platform: Perfect Root (AfricaHackon) **Room:** Echo Signature **Author:** LordSudo **Difficulty:** Medium **Points:** 200 **Category:** Smart Contract — ECDSA Signature Malleability / Replay **Result:** Solved (1/1 flag captured) — **FIRST BLOOD**

Description

> A relayer contract processes signed withdrawal authorizations from a single trusted signer. Logs show a withdrawal request was authorized once, but the recipient was paid twice — with no second signature ever collected. You're given the contract source and the exact request + signature that was already used once. No live instance and no private key are needed to reproduce the incident.

Flag

`r00t{a492e46c96231d82}`

Derivation: flag = "r00t{" + keccak256(r || (n-s) || v')[:16] + "}"

Where: - `n` = secp256k1 curve order - `r, s, v` = the signature from `used_signature.json` - `v' = 27` (flipped from the original `v = 28`) - Packing is raw bytes, big-endian, `r || s' || v'`, `v'` as a single byte - Truncate the keccak-256 digest to the first **16 hex chars** (8 bytes), lowercase, no `0x`

Vulnerability

ECDSA signature malleability enabling a **replay attack** (CWE-347 / SWC-117).

`WithdrawalRelayer` authenticates withdrawals via `ecrecover` and prevents replays by storing a hash of the signature tuple in `usedSignatures`. But the "identity" it hashes is `(r, s, v)` — which is not unique. Every valid ECDSA signature has a second, equally-valid representation `(r, n-s, vXOR1)` that recovers the same signer. The relayer accepts the second form as "new," and the recipient is paid twice.

Source Analysis

```
function executeWithdrawal(
    address recipient,
```

```
    uint256 amount,
    uint256 requestId,
    uint8 v,
    bytes32 r,
    bytes32 s
) external {
    bytes32 messageHash = keccak256(abi.encodePacked(recipient, amount, requestId));
    bytes32 ethSignedMessageHash = keccak256(
        abi.encodePacked("\x19Ethereum Signed Message:\n32", messageHash)
    );

    address signer = ecrecover(ethSignedMessageHash, v, r, s);
    require(signer == authorizedSigner, "invalid signature");

    bytes32 sigHash = keccak256(abi.encodePacked(r, s, v));
    require(!usedSignatures[sigHash], "signature already used");
    usedSignatures[sigHash] = true;

    paidOut[recipient] += amount;
    emit Withdrawn(recipient, amount, sigHash);
}
```

The bug in one line

```
bytes32 sigHash = keccak256(abi.encodePacked(r, s, v));
```

`(r, s, v)` is **not** a unique identifier for "this authorization was used." ECDSA has two valid forms per (message, signer). Hashing the tuple means the guard only blocks one of them.

ECDSA Malleability Math

Given a valid signature `(r, s, v)` over a message:

```
(r, n - s, v XOR 1)     where n = secp256k1 order
                        = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
```

recovers the same signer. `ecrecover` accepts both. Only the tuple hash differs.

From `used_signature.json`:

```
original:  s = 0x3310310d1c35bb4dbcbacb05478ad5d6f73f189a9f625db2a0764c842777310e
           v = 28

malleated: s' = n - s
              = 0xccefcef2e3ca44b2434534fab8752a27c36fc44c0fe642891f5c1208a8bf1033
           v' = 27
```

Local confirmation

Both forms recover the same authorized signer:

```
original  recovers: 0x473b73dfEDfcE7A3F18ad8180ca81772889797DB
malleated recovers: 0x473b73dfEDfcE7A3F18ad8180ca81772889797DB
```

Confirmed via `eth_account.Account.recover_message` using `encode_defunct(primitive=messageHash)` to reproduce the contract's exact EIP-191 pipeline.

Flag Derivation

The challenge asks for the flag from **the replay-check hash the contract computes** — i.e. the value stored in `usedSignatures` for the malleated signature. Not the message hash, not `ethSignedMessageHash`.

Exact steps

1. Take `r, s, v` from the provided signature.
2. Compute malleated values: s' = n - s v' = 27 (flip of 28)
3. Compute the sigHash **exactly as the contract would** on the replay: sigHash = keccak256(abi.encodePacked(bytes32 r, bytes32 s', uint8 v')) `abi.encodePacked` of these types is raw concatenation: `r || s' || v'` = 32 + 32 + 1 = 65 raw bytes, no padding, no length prefixes.
4. Truncate the hex digest to the first 16 hex chars (8 bytes), lowercase.
5. Wrap in `r00t{...}`.

Recompute

```
from eth_hash.auto import keccak

n  = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
r  = 0x3557278cc3fdf4ea20ef2d58d4cb891c5dadf772026e86568195eea6b86561af
s  = 0x3310310d1c35bb4dbcbacb05478ad5d6f73f189a9f625db2a0764c842777310e
v  = 28

s2 = n - s
v2 = 27

sig_hash = keccak(
    r.to_bytes(32, 'big') +
    s2.to_bytes(32, 'big') +
    bytes([v2])
).hex()

flag = "r00t{" + sig_hash[:16] + "}"
```

Output:

```
sigHash  = a492e46c96231d82708688031fe72538391b3bd1451c47d4a642890543cbc273
first 16 = a492e46c96231d82
flag     = r00t{a492e46c96231d82}
```

What Worked

1. **Reading the contract literally.** The replay guard's failure mode (`usedSignatures[keccak(r,s,v)]`) is visible in two lines of Solidity.

2. **Knowing ECDSA malleability.** `(r, s, v)` ↔ `(r, n-s, vXOR1)`. Both recover the same signer.
3. **Confirming locally with `eth_account`.** Both forms recovering the same address proved the replay was real, not theoretical.
4. **Asking the author for a directional nudge** after exhausting every reasonable derivation. This is the single highest-leverage move on a medium challenge you're stuck on. "Is the flag from the selector, the signature material, or a phrase?" — one sentence, unblocked everything.
5. **Listening precisely.** When the author said "the malleated v, packed r||s||v as raw bytes, truncated to 16 hex," applying that exactly landed the flag.

What Failed (and Why)

1. **Guessing flag formats.** We submitted selectors, full hashes, 4-byte truncations, descriptive phrases, and header values. Dozens of submissions. Zero landed.

   Lesson: don't guess when the challenge has a *derivable* answer. Compute, don't conjecture.

2. **Assuming the same truncation as a similar challenge.** "The Vault That Ate Itself" used the 4-byte selector. We assumed Echo Signature would too. It used an 8-byte (16 hex) truncation of a keccak hash instead.

   Lesson: similar wording ≠ identical derivation. Generate candidates at *every* truncation length — 4, 8, 16, 32 hex — before submitting.

3. **Using the original `v = 28`.** We computed the sigHash with the original signature the first time. The flag is derived from the **malleated** signature — the one that would actually be replayed. The contract stores the malleated hash on the replay; that's the value the flag encodes.

   Lesson: on a replay challenge, the flag encodes the *attacker's* inputs (the replayed signature), not the legitimate one.

4. **Not reading the challenge description precisely enough.** "Derive the flag from the vulnerable function's own signature" — this could mean many things. For Vault it meant the selector; here it meant the replay-check hash. The phrase is deliberately ambiguous and does not carry the derivation by itself.

5. **Very long heredocs with triple-backtick code fences.** zsh parses backticks as command substitution and swallows the heredoc terminator, leaving the shell at `heredoc>`. Multiple writeup attempts failed this way.

   Lesson: write markdown containing code fences with `nano` or `vim`, or use `~~~` fences instead of triple backticks.

Real-World Impact

Signature replay on bridges and relays

Cross-chain bridges and meta-transaction relayers (ERC-2771, EIP-712-based forwarders) routinely accept off-chain signatures from a trusted signer or user. If they identify "a used signature" by hashing `(r, s, v)` instead of hashing the *message*, an attacker can replay indefinitely.

Historical cases:

- **Poly Network (2021, ~$610M)** — signature verification logic flaw.

- **Wormhole (2022, ~$326M)** — signature verification bypass.
- **Nomad (2022, ~$190M)** — replay of a valid root message.
- **Various ERC-2771 forwarders** — replayed meta-transactions when nonces were missing from the signed payload.

The specific failure mode

Any system that uses `keccak256(r || s || v)` (or `abi.encodePacked(r, s, v)`) as the "used" identifier is **broken by construction**. It will always be malleable unless the code additionally rejects high-S signatures via `require(uint256(s) <= n / 2)`.

How It Happens in Real Codebases

1. **"Signature hash" as identity.** Developers naturally think of the signature bytes as a unique fingerprint of an authorization. They aren't. The *message* is the identity; the signature is one of many encodings.
2. **Copying examples that hash (r, s, v).** Many older tutorials use `keccak256(abi.encodePacked(r, s, v))` because it works for the happy path and looks reasonable.
3. **No low-S enforcement.** Solidity's `ecrecover` accepts both malleable forms. EIP-2 rejects high-S at the *transaction signature* layer, but not in application-level `ecrecover` calls. OpenZeppelin's `ECDSA.recover` rejects high-S explicitly — many hand-rolled versions don't.
4. **No nonce in the signed payload.** If the signed message includes a per-user monotonic nonce, replay is structurally impossible even if the signature tuple is malleated. Many contracts omit this.
5. **No `chainId` or contract address in the signed message.** Cross-chain and cross-contract replay become possible even without malleability.

Fixes (Defensive)

1. Mark by messageHash, not sigHash

The simplest correct fix:

```
bytes32 messageHash = keccak256(abi.encodePacked(recipient, amount, requestId));
require(!usedMessages[messageHash], "authorization already used");
usedMessages[messageHash] = true;
```

The message is unique per authorization; the signature is not.

2. Include a per-user nonce in the signed payload

```
bytes32 messageHash = keccak256(
    abi.encodePacked(recipient, amount, nonces[recipient]++, requestId)
);
```

Even a malleated signature cannot be replayed because the nonce advances on first use.

3. Use OpenZeppelin's ECDSA library

```
import "@openzeppelin/contracts/utils/cryptography/ECDSA.sol";

address signer = ECDSA.recover(ethSignedMessageHash, v, r, s);
```

`ECDSA.recover` rejects `s > n / 2` (high-S). This kills malleability at the root: there is only one valid signature per message.

4. Adopt EIP-712 typed structured data

EIP-712 signatures include a domain separator (`name`, `version`, `chainId`, `verifyingContract`). This prevents cross-contract and cross-chain replay as a side effect of a stronger domain definition.

5. Add a deadline to the signed payload

```
bytes32 messageHash = keccak256(
    abi.encodePacked(recipient, amount, requestId, deadline)
);
require(block.timestamp <= deadline, "expired");
```

Limits the blast radius if replay is possible through any other vector.

6. Prefer a commit-reveal or on-chain nonce over signature-based auth

If the authorization doesn't need to be off-chain, don't make it signature- based. Each simplification removes an attack surface.

Key Takeaways

- **Signature ≠ identity.** The message is the identity. Never key replay protection off the signature tuple.
- **ECDSA malleability is a property of the algorithm, not a bug in your code.** `(r, s, v)` and `(r, n-s, vXOR1)` are both valid. Every contract that hashes `(r, s, v)` as an identity is exploitable.
- **EIP-2 does not protect you.** It bans high-S at the transaction layer only. Application `ecrecover` calls accept both.
- **Use OZ `ECDSA.recover`.** It rejects high-S. Use it. Don't hand-roll.
- **Nonce > signature-hash.** A per-user nonce makes replay structurally impossible regardless of malleability.
- **Read the challenge description literally.** "Replay-check hash" means the value stored in `usedSignatures` for the replayed signature — the malleated one — not the legitimate one, and not the message hash.
- **Ask the author when stuck.** After exhausting derivations, one directional nudge from a challenge author is legitimate and efficient. Rule 03 forbids spoilers from other players; asking for a *direction* from the author is not a spoiler.

Tools Used

- `eth_hash` (Python) — keccak-256 for hash and selector derivation
- `eth_account` — EIP-191 aware `ecrecover` for local confirmation
- Foundry `cast sig` — independent selector cross-check
- Plain source review

Commands That Worked

```
# compute the sigHash the contract would store on the replay
python3 - << 'PY'
from eth_hash.auto import keccak
n = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141
r = 0x3557278cc3fdf4ea20ef2d58d4cb891c5dadf772026e86568195eea6b86561af
s = 0x3310310d1c35bb4dbcbacb05478ad5d6f73f189a9f625db2a0764c842777310e
v = 28
s2, v2 = n - s, 27
h = keccak(r.to_bytes(32,'big') + s2.to_bytes(32,'big') + bytes([v2])).hex()
print("r00t{" + h[:16] + "}")
PY
```

Commands That Did NOT Work (and why)

```
# Long heredoc with triple-backtick fences - zsh eats the terminator
cat > writeup.md << 'EOF'
...
```solidity
...
EOF
# Fix: use nano/vim, or use ~~~ fences instead of ```

# Full 64-hex sigHash as flag - never tried the 16-hex truncation
r00t{a492e46c96231d82708688031fe72538391b3bd1451c47d4a642890543cbc273}
# Fix: truncate to first 16 hex chars

# 4-byte selector - pattern borrowed from "The Vault That Ate Itself"
r00t{346b4263}
# Fix: different challenge, different derivation

# Original signature (v = 28, s as given) - wrong form
keccak(r || s || v = 28)
# Fix: use malleated values
```
```